

ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ ПРОЕКТА

TCP checkers

Настольное приложение «Шашки по сети»

Оглавление

1. Общая характеристика проекта	4
1.1. Основные функции	4
1.2. Рабочие состояния	5
2. Технологический стек	5
3. Структура проекта	6
4. Архитектура и взаимодействие компонентов.....	7
4.1. Последовательность запуска партии	7
4.2. Синхронизация состояния.....	8
5. Игровая модель и правила	8
5.1. Доска и фигуры	8
5.2. Проверка и выполнение хода.....	9
5.3. Определение победителя.....	9
5.4. Доменные модели	10
6. Пользовательский интерфейс и сценарии	10
6.1. Состав главного окна.....	10
6.2. Отображение доски.....	11
6.3. Управление партией	11
7. Сетевой протокол.....	11
7.1. Транспорт и формат.....	11
7.2. Типы сообщений	12
7.3. Поля NetworkMessage.....	12
7.4. Обработка соединения.....	13
8. Конфигурация, публикация и установка	13

8.1. Основной проект	13
8.2. Профиль публикации FolderProfile	14
8.3. Сценарий установщика	14
9. Зависимости проекта	15
9.1. Основное приложение	15
9.2. Тестовый проект.....	16
10. Инструкция по локальному запуску	16
10.1. Требования к окружению.....	16
10.2. Сборка и запуск из исходников.....	16
10.3. Запуск партии	17
10.4. Запуск готовой публикации и установщика	17
11. Сборка, тестирование и публикация	17
11.1. Команды.....	17
11.2. Тестовое покрытие.....	18
12. Особенности сопровождения и расширения.....	18

1. Общая характеристика проекта

TSP checkers — настольное приложение для проведения партии в шашки между двумя участниками через TCP-соединение. Один экземпляр приложения запускает сервер и играет белыми, второй экземпляр подключается к серверу и играет черными.

Каждый участник выполняет игровую логику локально. Синхронизация партии выполняется передачей ходов и служебных сообщений в JSON. Интерфейс отображает доску, состояние соединения, назначенный цвет, текущего игрока, состояние партии и журнал событий.

1.1. Основные функции

- запуск TCP-сервера на выбранном порту и ожидание одного подключения;
- подключение клиента по числовому IP-адресу и порту;
- отображение доски 8 на 8 и начальной расстановки из 12 белых и 12 черных шашек;
- проверка допустимости ходов, обязательных взятий и последовательных взятий одной фигурой;
- поддержка обычных шашек и дамок, включая превращение на крайнем ряду;
- определение победителя по количеству фигур и наличию допустимых ходов;
- разворот доски для участника, играющего черными;
- обработка завершения партии, запуска новой партии и отключения участника;
- отображение сообщений приложения с отметкой времени в формате HH:mm:ss.

1.2. Рабочие состояния

Состояние	Назначение
WaitingForConnection	Доска подготовлена; приложение ожидает установления и подтверждения соединения.
Playing	Партия начата; ходы разрешены участнику, цвет которого совпадает с текущим игроком.
WhiteWon	Партия завершена победой белых.
BlackWon	Партия завершена победой черных.
ConnectionError	Соединение завершено пользователем или потеряно во время партии.

2. Технологический стек

Категория	Технология	Назначение
Язык	C#	Реализация интерфейса, игровой логики, моделей и сетевого взаимодействия.
Платформа	.NET 8	Среда выполнения, сборка, запуск, тестирование и публикация.
Интерфейс	Windows Forms	Настольное окно, элементы управления и игровая доска.
Сеть	System.Net, System.Net.Sockets	TcpListener, TcpClient и NetworkStream для обмена между двумя экземплярами приложения.
Сериализация	System.Text.Json	Преобразование NetworkMessage в JSON и обратно.
Графика	System.Drawing	Загрузка, масштабирование и отображение изображений фигур.
Сборка	MSBuild, dotnet CLI	Восстановление зависимостей, сборка, запуск, тестирование и публикация.
Тестирование	xUnit, Microsoft.NET.Test.Sdk	Модульные и сетевые loopback-тесты.
Публикация	dotnet publish	Файловая self-contained публикация для win-x64.
Установщик	Inno Setup	Формирование Windows-установщика CheckersSetup.exe.

3. Структура проекта

Путь	Назначение
README.md	Описание назначения, функций, стека, команд запуска, публикации и тестирования.
project/project.slnx	Файл решения, включающий основное приложение и тестовый проект.
project/project.csproj	Конфигурация основного Windows Forms-приложения.
project/Program.cs	Точка входа: инициализация Windows Forms и запуск MainForm.
project/MainForm.cs	Координация интерфейса, игровой логики и сетевых компонентов.
project/MainForm.Designer.cs	Определение элементов управления и компоновки главного окна.
project/MainForm.resx	Ресурсы формы Windows Forms.
project/GameLogic/	Игровая доска, правила ходов и управление состоянием партии.
project/Models/	Доменные модели, перечисления и контракт сетевых сообщений.
project/Network/	Асинхронные TCP-сервер и TCP-клиент.
project/Properties/AssemblyInfo.cs	Открытие внутренних членов основного проекта для project.Tests.
project/Properties/PublishProfiles/FolderProfile.pubxml	Профиль файловой self-contained публикации.
project.Tests/	Тесты игровой логики, сериализации и TCP-взаимодействия.
installer/CheckersSetup.iss	Сценарий формирования установщика Inno Setup.
installer/CheckersSetup.exe	Готовый установщик приложения.
publish/	Готовая self-contained публикация с исполняемым файлом project.exe.

4. Архитектура и взаимодействие компонентов

Приложение построено как единый Windows Forms-процесс. MainForm создает экземпляры CheckersGame, TcpGameServer и TcpGameClient, подписывается на их события и управляет пользовательскими сценариями. Игровой движок не обращается к элементам интерфейса и сетевым классам.

Компонент	Ответственность	Связи
Program	Инициализация приложения и запуск MainForm.	Передаёт управление Windows Forms.
MainForm	Интерфейс, выбор режима, ввод адреса и порта, обработка ходов, журнал сообщений.	Вызывает CheckersGame и выбранный сетевой компонент; обрабатывает события.
CheckersGame	Состояние партии, применение ходов, смена игрока, победитель и последовательное взятие.	Использует GameBoard и MoveValidator; публикует BoardChanged и GameStateChanged.
GameBoard	Хранение массива Piece?[8,8], начальная расстановка, изменение клеток и снимок доски.	Используется CheckersGame и MoveValidator.
MoveValidator	Проверка ходов и построение списка доступных ходов.	Читает текущее состояние GameBoard.
TcpGameServer	Прослушивание порта, прием одного клиента, отправка и получение строк JSON.	Публикует ClientConnected, MessageReceived и ConnectionLost.
TcpGameClient	Подключение к серверу, отправка и получение строк JSON.	Публикует Connected, MessageReceived и ConnectionLost.
Models	Координаты, ходы, фигуры, результаты, состояния и сетевые сообщения.	Используются всеми функциональными слоями.

4.1. Последовательность запуска партии

1. Серверный экземпляр проверяет порт, назначает локальному участнику белый цвет, сбрасывает партию и запускает TcpListener на IPAddress.Any.

2. Клиентский экземпляр проверяет числовой IP-адрес и порт, назначает локальному участнику черный цвет и выполняет `TcpClient.ConnectAsync`.
3. После подключения клиент отправляет сообщение `connect` с `PlayerColor.Black`.
4. Сервер получает `connect`, переводит `CheckersGame` в состояние `Playing` и отправляет ответный `connect` с `PlayerColor.White`.
5. Клиент получает ответный `connect` и переводит локальный `CheckersGame` в состояние `Playing`.
6. Белые выполняют первый ход. Каждый успешный ход применяется локально и передается второму экземпляру сообщением `move`.

4.2. Синхронизация состояния

Оба экземпляра приложения создают одинаковую начальную доску и применяют одинаковую последовательность ходов через `CheckersGame.TryMakeMove`. Полученный от второго участника ход проходит ту же проверку правил, что и локальный ход. Некорректный удаленный ход не изменяет доску и фиксируется в журнале интерфейса.

После успешного хода `CheckersGame` публикует `BoardChanged`. Изменение состояния партии публикуется через `GameStateChanged`. `MainForm` обрабатывает события в UI-потокe через `InvokeRequired` и `BeginInvoke`.

5. Игровая модель и правила

5.1. Доска и фигуры

- `GameBoard` хранит двумерный массив `Piece?[8,8]`.
- Начальная расстановка занимает темные клетки: черные находятся в строках 0–2, белые — в строках 5–7.
- `Piece` содержит `Color` типа `PlayerColor` и `Type` типа `PieceType`.

- PlayerColor содержит значения White и Black; PieceType содержит значения Man и King.
- GetSnapshot создает новый массив и клонирует каждую фигуру, поэтому интерфейс получает снимок без прямого доступа к внутреннему массиву.

5.2. Проверка и выполнение хода

MoveValidator проверяет границы доски, наличие фигуры, принадлежность текущему игроку, пустую целевую клетку и диагональное направление. При наличии хотя бы одного взятия простой ход блокируется.

Фигура	Простой ход	Взятие
Обычная шашка	Одна клетка по диагонали вперед: белые уменьшают номер строки, черные увеличивают.	Прыжок на две клетки по любой диагонали через фигуру соперника.
Дамка	Любое количество свободных клеток по диагонали.	Перемещение по диагонали через одну фигуру соперника на любую свободную клетку после нее.

После взятия движок проверяет дополнительное взятие той же фигурой. Поле requiredCaptureFrom фиксирует обязательную исходную клетку следующего хода. Смена игрока выполняется после завершения всей последовательности взятий.

Обычная шашка превращается в дамку при достижении строки 0 для белых или строки 7 для черных. Превращение выполняется до проверки дополнительного взятия.

5.3. Определение победителя

CheckersGame.CheckWinner возвращает цвет победителя, когда у соперника закончились фигуры либо текущий после смены хода игрок не имеет допустимых ходов. Состояние партии переводится в WhiteWon или BlackWon, после чего интерфейс блокирует игровую доску.

5.4. Доменные модели

Модель	Поля и назначение
CellPosition	Row, Col; координата клетки, проверка диапазона 0–7 и сравнение координат.
Move	FromRow, FromCol, ToRow, ToCol; вычисление смещения и проверка диагонали.
Piece	Color, Type; определение дамки, превращение и клонирование.
MoveResult	Success, Message, CapturedPiece, Winner, IsCapture, RequiresAdditionalCapture.
GameState	WaitingForConnection, Playing, WhiteWon, BlackWon, ConnectionError.
NetworkMessage	Плоский JSON-контракт служебных сообщений и ходов.

6. Пользовательский интерфейс и сценарии

6.1. Состав главного окна

Главное окно MainForm имеет размер 1252 на 700 и минимальный размер 1268 на 739. Верхняя панель содержит поля IP-адреса и порта, а также кнопки «Создать сервер», «Подключиться», «Отключиться» и «Новая игра». Центральная область содержит игровую доску и панель состояния.

Элемент	Поведение
IP-адрес	Начальное значение 127.0.0.1; клиент принимает адрес, успешно проверяемый IPAddress.TryParse.
Порт	Начальное значение 5000; допустимый диапазон от 1 до 65535.
Игровая доска	64 кнопки в TableLayoutPanel; активна только во время хода локального участника.
Панель состояния	Показывает текущего игрока, локальный цвет, состояние соединения и состояние партии.
Журнал сообщений	Добавляет события с локальным временем в формате HH:mm:ss и прокручивается к последней записи.

6.2. Отображение доски

Клетки окрашиваются цветами SaddleBrown и Bisque; выбранная клетка выделяется цветом Gold. Для черного участника доска разворачивается преобразованием координат $row \rightarrow 7 - row$ и $col \rightarrow 7 - col$.

Фигуры отображаются четырьмя PNG-изображениями, которые копируются в выходной каталог согласно project.csproj и масштабируются методом ResizeCheckerImage. При ошибке загрузки применяется текстовое отображение W, B, WK и BK.

6.3. Управление партией

- Выбор фигуры разрешен только для локального цвета и только во время хода локального участника.
- Повторный выбор выделенной клетки снимает выделение.
- После первого хода последовательного взятия выбранной остается фигура, которая должна продолжить взятие.
- Кнопка «Новая игра» при активном соединении запускает новую партию после завершения текущей и отправляет сообщение newGame.
- Кнопка «Отключиться» отправляет сообщение disconnect, останавливает сетевые компоненты и переводит игру в ConnectionError.
- Заккрытие формы останавливает TCP-компоненты и освобождает изображения фигур.

7. Сетевой протокол

7.1. Транспорт и формат

TcpGameServer и TcpGameClient используют NetworkStream, StreamReader и StreamWriter с кодировкой UTF-8. Каждое сообщение сериализуется в одну

строку JSON и отправляется методом WriteLineAsync; прием выполняется методом ReadLineAsync.

NetworkMessage использует JsonSerializerOptions с сохранением явно заданных имен свойств и пропуском полей со значением null. Сервер привязывается к IPAddress.Any и принимает одно TCP-подключение за один запуск Start.

7.2. Типы сообщений

type	Поля	Назначение
connect	playerColor	Подтверждение подключения и назначенного цвета; запускает связанную партию.
move	fromRow, fromCol, toRow, toCol	Передача выполненного хода второму участнику.
gameOver	winner	Уведомление о победителе после завершающего хода.
newGame	только type	Синхронный запуск новой партии после завершения предыдущей.
error	errorText	Передача текстового описания сетевой ошибки.
disconnect	reason	Уведомление о завершении соединения.

7.3. Поля NetworkMessage

JSON-поле	Тип C#	Назначение
type	string	Диспетчеризация сообщения в MainForm.OnMessageReceived.
fromRow	int?	Исходная строка хода.
fromCol	int?	Исходный столбец хода.
toRow	int?	Целевая строка хода.
toCol	int?	Целевой столбец хода.
winner	PlayerColor?	Цвет победителя.

JSON-поле	Тип C#	Назначение
errorText	string?	Текст сообщения error.
playerColor	PlayerColor?	Цвет участника в сообщении connect.
reason	string?	Причина отключения.

7.4. Обработка соединения

- TcpGameServer.Start сначала останавливает ранее созданные ресурсы, затем запускает TcpListener и ожидание клиента.
- TcpGameClient.Connect сначала отключает ранее созданные ресурсы, затем открывает новое соединение.
- Получение сообщений выполняется в асинхронном цикле до отмены CancellationTokenSource или потери соединения.
- SocketException, IOException и ObjectDisposedException приводят к событию потери соединения.
- MainForm при потере соединения останавливает сетевые компоненты, снимает выделение, активирует кнопки выбора режима и блокирует игровую доску.
- Десериализация входящей строки требует корректного JSON; JsonException сетевыми классами не перехватывается.

8. Конфигурация, публикация и установка

8.1. Основной проект

Параметр project.csproj	Значение	Назначение
OutputType	WinExe	Сборка графического Windows-приложения.
TargetFramework	net8.0-windows	Целевая платформа .NET 8 для Windows.
UseWindowsForms	true	Подключение Windows Forms.

Параметр project.csproj	Значение	Назначение
Nullable	enable	Включение nullable reference types.
ImplicitUsings	enable	Автоматическое подключение стандартных пространств имен.
ApplicationIcon	app.ico	Иконка исполняемого файла.
Content	четыре PNG-файла фигур	Копирование ресурсов в выходной подкаталог с режимом PreserveNewest.

8.2. Профиль публикации FolderProfile

Параметр	Значение
Configuration	Release
Platform	Any CPU
PublishProtocol	FileSystem
TargetFramework	net8.0-windows
RuntimeIdentifier	win-x64
SelfContained	true
PublishSingleFile	false
PublishReadyToRun	false
PublishDir	C:\Users\isoko\Documents\TCP checkers\publish

Self-contained публикация включает runtime .NET и запускается через publish\project.exe. Параметр PublishDir содержит абсолютный путь текущего рабочего каталога.

8.3. Сценарий установщика

Параметр	Значение
Сценарий	installer/CheckersSetup.iss
Имя приложения	Шашки по сети
Версия	1.0

Параметр	Значение
Исполняемый файл	project.exe
Архитектура	x64compatible; установка в 64-битном режиме
Права	PrivilegesRequired=lowest
Языки установщика	english, russian
Выходной файл	installer/CheckersSetup.exe
Ярлыки	меню программ; ярлык рабочего стола как необязательная задача
Завершение установки	предложение запустить project.exe

Сценарий копирует project.exe и остальные файлы публикации в каталог приложения. Пути OutputDir, SetupIconFile и Source заданы абсолютными путями внутри рабочего каталога; для сборки из другого расположения они должны соответствовать фактическому пути репозитория.

9. Зависимости проекта

9.1. Основное приложение

Зависимость	Назначение
.NET 8 Windows Desktop Runtime	Выполнение Windows Forms-приложения и системных библиотек.
System.Windows.Forms	Главное окно, панели, кнопки, поля ввода, метки, список сообщений и TableLayoutPanel.
System.Net / System.Net.Sockets	TCP-сервер и TCP-клиент.
System.Text.Json	JSON-контракт сетевых сообщений.
System.Drawing	Иконка приложения и изображения фигур.

9.2. Тестовый проект

Пакет	Версия	Назначение
Microsoft.NET.Test.Sdk	17.11.1	Запуск тестов через dotnet test и интеграция с тестовой платформой.
xunit	2.9.2	Фреймворк тестов и утверждений.
xunit.runner.visualstudio	2.8.2	Адаптер запуска xUnit; указан как PrivateAssets=all.
ProjectReference: project.csproj	текущий проект	Доступ к игровой логике, моделям и сетевым классам.

10. Инструкция по локальному запуску

10.1. Требования к окружению

- операционная система Windows;
- .NET SDK 8.0 для запуска из исходников;
- доступный TCP-порт в диапазоне 1–65535;
- два экземпляра приложения для проведения сетевой партии.

10.2. Сборка и запуск из исходников

7. Открыть PowerShell в корне репозитория.

8. Собрать основной проект командой:

```
dotnet build .\project\project.csproj
```

9. Запустить первый экземпляр приложения командой:

```
dotnet run --project .\project\project.csproj
```

10. Запустить второй экземпляр той же командой на локальном или другом Windows-компьютере.

10.3. Запуск партии

11. В серверном экземпляре указать порт и нажать «Создать сервер». Сервер прослушивает все локальные сетевые интерфейсы и назначает участнику белый цвет.

12. В клиентском экземпляре указать числовой IP-адрес сервера и тот же порт, затем нажать «Подключиться». Клиент получает черный цвет.

13. Дождаться сообщения «Подключение выполнено. Игра начинается.» и состояния «идет партия».

14. Белые выполняют первый ход. Интерфейс активирует доску только для участника, которому принадлежит текущий ход.

Для запуска двух экземпляров на одном компьютере используется адрес 127.0.0.1. Начальное значение порта в интерфейсе — 5000.

10.4. Запуск готовой публикации и установщика

- Self-contained публикация запускается файлом `publish\project.exe`.
- Готовый установщик запускается файлом `installer\CheckersSetup.exe`.

11. Сборка, тестирование и публикация

11.1. Команды

Операция	Команда
Сборка приложения	<code>dotnet build .\project\project.csproj</code>
Запуск приложения	<code>dotnet run --project .\project\project.csproj</code>
Запуск тестов	<code>dotnet test .\project.Tests\project.Tests.csproj</code>
Публикация	<code>dotnet publish .\project\project.csproj -c Release -r win-x64 --self-contained true -p:PublishProfile=FolderProfile</code>
Сборка установщика	<code>ISCC .\installer\CheckersSetup.iss</code>

11.2. Тестовое покрытие

Тестовый проект `project.Tests` содержит 18 тестов. Контрольная сборка основного проекта завершается без предупреждений и ошибок; полный запуск тестового проекта завершает 18 из 18 тестов успешно.

Класс тестов	Количество	Проверяемые сценарии
GameLogicTests	10	Начальная расстановка, состояния партии, неизменность доски при ошибочном ходе, обязательное и последовательное взятие, превращение в дамку, победа по фигурам и доступным ходам.
NetworkTests	8	Сериализация <code>move</code> , <code>gameOver</code> , <code>error</code> , <code>disconnect</code> , <code>connect</code> и <code>newGame</code> ; передача <code>move</code> через <code>loopback</code> ; событие потери соединения.

`NetworkTests` получает свободный `loopback`-порт через `TcpListener` на порту 0 и ограничивает ожидание сетевых событий пятью секундами.

12. Особенности сопровождения и расширения

- `MainForm` является точкой координации интерфейса, игрового движка и сетевого транспорта; новые пользовательские сценарии подключаются через обработчики элементов управления и событий.
- `CheckersGame` инкапсулирует переходы состояния партии и публикует события изменения; расширение правил выполняется совместно с `MoveValidator` и тестами `GameLogicTests`.
- `GameBoard.GetSnapshot` возвращает копии фигур, что сохраняет владение изменяемым состоянием внутри игрового движка.
- Внутренний конструктор `CheckersGame` принимает подготовленную доску и состояние; доступ тестового проекта обеспечивается атрибутом `InternalsVisibleTo("project.Tests")`.

- Синхронизация основана на детерминированном применении последовательности ходов обоими участниками; изменение правил требует одинаковой версии приложения на обеих сторонах соединения.
- Сетевой контракт централизован в NetworkMessage; добавление нового типа сообщения требует фабрики или заполнения полей, обработки в MainForm.OnMessageReceived и тестов сериализации.
- TcpGameServer принимает одного клиента за один запуск Start. Новый цикл ожидания создается повторным запуском серверного режима.
- TcpGameClient принимает только числовой IP-адрес, поскольку интерфейс проверяет ввод через IPAddress.TryParse.
- Файлы публикационного профиля и сценария установщика содержат абсолютные пути текущего рабочего каталога.
- Операции запуска сервера и подключения клиента отображают пользователю общие сообщения об ошибке без текста исключения.